List Data Structures

In [1]:
```
l = []
l
```

Out[1]:
```
[]
```

In [4]:
```
l.append(10)
l.append(10)
l.append(10)
l
```

Out[4]:
```
[10, 10, 10]
```

In [5]:
```
l.append(20)
l.append(30)
l
```

Out[5]:
```
[10, 10, 10, 20, 30]
```

In [6]:
```
type(l)
```

Out[6]:
```
list
```

In [7]:
```
l1 = [10 , '2.3', 1+2j, True, 'krith', [1,2,3,4]]
l1
```

Out[7]:
```
[10, '2.3', (1+2j), True, 'krith', [1, 2, 3, 4]]
```

user can define multiple data types --> that's why is home genious & hetorogenious allowed

In [8]:
```
len(l1)
```

Out[8]:
```
6
```

In [9]:
```
print(l)
print(l1)
```

```
[10, 10, 10, 20, 30]
[10, '2.3', (1+2j), True, 'krith', [1, 2, 3, 4]]
```

In [11]:
```
print(id(l))
print(id(l1))
```

```
2936752077696
2936799685568
```

In [12]:
```
l2 = [10,20,30,40,50,10]
l2
```

Out[12]:
```
[10, 20, 30, 40, 50, 10]
```

In [13]:
```
len(l2)
```

Out[13]:
```
6
```

In [14]:
```
l2.clear()
```

In [15]:
```
len(l2)
```

Out[15]:
```
0
```

In [16]:
```
l2
```

Out[16]:
```
[]
```

In [17]:
```
del l2
l2
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[17], line 2
      1 del l2
----> 2 l2

NameError: name 'l2' is not defined
```

In [18]:
```
l.count(20)
```

Out[18]:
```
1
```

In [19]:
```
l.count(10)
```

Out[19]:
```
3
```

In [20]:
```
l.count(100)
```

Out[20]:
```
0
```

In [22]:
```
a = [1,2,3]
b = [4,5,6]
a.extend(b)
a
```

Out[22]:
```
[1, 2, 3, 4, 5, 6]
```

In [23]:
```
l2 = []
l2
```

Out[23]: []

In [25]:
```
l2.extend(l1)
l2
```

Out[25]: [10, 10, 10, 20, 30, 10, '2.3', (1+2j), True, 'krith', [1, 2, 3, 4]]

In [26]:
```
print(l)
print(l1)
print(l2)
```

```
[10, 10, 10, 20, 30]
[10, '2.3', (1+2j), True, 'krith', [1, 2, 3, 4]]
[10, 10, 10, 20, 30, 10, '2.3', (1+2j), True, 'krith', [1, 2, 3, 4]]
```

In [27]:
```
l[0]
```

Out[27]: 10

In [ ]:
```
# list is mutable
```

In [28]:
```
l[0] = 100
l
```

Out[28]: [100, 10, 10, 20, 30]

In [30]:
```
job = [5000]
job[0]
```

Out[30]: 5000

In [31]:
```
job[0] = 20000
job
```

Out[31]: [20000]

In [32]:
```
l[3] = 25
l[3]
```

Out[32]: 25

In [33]:
```
l
```

Out[33]: [100, 10, 10, 25, 30]

In [34]:
```
l[-2] = 600
l
```

Out[34]: [100, 10, 10, 600, 30]

In [35]:
```
l.index(600)
```

Out[35]: 3

In [36]:
```
l2
```

Out[36]: [10, 10, 10, 20, 30, 10, '2.3', (1+2j), True, 'krith', [1, 2, 3, 4]]

In [38]:
```
l2.index(10)
```

Out[38]: 0

In [39]:
```
l
```

Out[39]: [100, 10, 10, 600, 30]

In [40]:
```
l.insert(2,25)
l
```

Out[40]: [100, 10, 25, 10, 600, 30]

In [41]:
```
l
```

Out[41]: [100, 10, 25, 10, 600, 30]

In [42]:
```
l2
```

Out[42]: [10, 10, 10, 20, 30, 10, '2.3', (1+2j), True, 'krith', [1, 2, 3, 4]]

In [43]:
```
l2.pop()
```

Out[43]: [1, 2, 3, 4]

In [ ]:
```
# pop function will remove last element by default
```

In [44]:
```
l2.pop()
```

Out[44]: 'krith'

In [45]:
```
l2
```

Out[45]: [10, 10, 10, 20, 30, 10, '2.3', (1+2j), True]

In [46]:
```
l2.pop(2)
```

Out[46]: 10

In [47]:
```
l2
```

Out[47]: [10, 10, 20, 30, 10, '2.3', (1+2j), True]

In [ ]:
```
# pop(2) -- 2nd index value will be eleminated
# remove == remove will removed by value
```

```
# pop vs remove --- pop is removed by index and remove is removed by value.
# by default pop will remove last element if we didn't mention any argument
```

In [48]:
```
l2.remove(30)
```

In [49]:
```
l2
```

Out[49]:
```
[10, 10, 20, 10, '2.3', (1+2j), True]
```

In [50]:
```
l2.reverse()
l2
```

Out[50]:
```
[True, (1+2j), '2.3', 10, 20, 10, 10]
```

In [51]:
```
l
```

Out[51]:
```
[100, 10, 25, 10, 600, 30]
```

In [52]:
```
l.sort()
```

In [53]:
```
l
```

Out[53]:
```
[10, 10, 25, 30, 100, 600]
```

In [54]:
```
l.sort(reverse=True)
```

In [55]:
```
l
```

Out[55]:
```
[600, 100, 30, 25, 10, 10]
```

In [56]:
```
l3 = [10,20,[20,30],30]
l3
```

Out[56]:
```
[10, 20, [20, 30], 30]
```

In [60]:
```
l3[2] = [20]
l3
```

Out[60]:
```
[10, 20, [20], 30]
```

In [61]:
```
l3 = [3,100,2,50]
l3
```

Out[61]:
```
[3, 100, 2, 50]
```

In [62]:
```
l3.sort()
```

In [63]:
```
l3
```

Out[63]:
```
[2, 3, 50, 100]
```

In [ ]:
```
# parameter tuning -- window 11 is in build (parameter tuning)
# hyperparameter tuning -- user remove system parameter wind 10 to window 11 -- hyp
```

In [64]:
```
l3.sort(reverse = True)
l3
```

Out[64]:
```
[100, 50, 3, 2]
```

In [65]:
```
l4 = [2,'a',2.4,True]
l4.sort()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[65], line 2
      1 l4 = [2,'a',2.4,True]
----> 2 l4.sort()

TypeError: '<' not supported between instances of 'str' and 'int'
```

In [66]:
```
l5 = ['z','b','m','c']
l5.sort()
```

In [67]:
```
l5
```

Out[67]:
```
['b', 'c', 'm', 'z']
```

In [68]:
```
l5.sort(reverse = True)
l5
```

Out[68]:
```
['z', 'm', 'c', 'b']
```

In [ ]:
```
# indexing & slicing in DS
```

In [69]:
```
l1[:]
```

Out[69]:
```
[10, '2.3', (1+2j), True, 'krith', [1, 2, 3, 4]]
```

In [70]:
```
l1[::-2]
```

Out[70]:
```
[[1, 2, 3, 4], True, '2.3']
```

In [71]:
```
l1
```

Out[71]:
```
[10, '2.3', (1+2j), True, 'krith', [1, 2, 3, 4]]
```

In [72]:
```
l1[0:10:3]
```

Out[72]:
```
[10, True]
```

In [73]:
```
l1
```

Out[73]:
```
[10, '2.3', (1+2j), True, 'krith', [1, 2, 3, 4]]
```

In [74]:
```
l1[2] = 45
l1
```

Out[74]:
```
[10, '2.3', 45, True, 'krith', [1, 2, 3, 4]]
```

In [ ]:
```
# list ds are completed
# tuple
```

In [75]:
```
t = ()
t
```

Out[75]:
```
()
```

In [76]:
```
type(t)
```

Out[76]:
```
tuple
```

In [77]:
```
t1 = (10,20,30)
t1
```

Out[77]:
```
(10, 20, 30)
```

In [78]:
```
t1.count(10)
```

Out[78]:
```
1
```

In [79]:
```
t1.count(100)
```

Out[79]:
```
0
```

In [80]:
```
t1.index(10)
```

Out[80]:
```
0
```

In [81]:
```
t1.index(30)
```

Out[81]:
```
2
```

In [82]:
```
t1[1] = 40
t1
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[82], line 1
----> 1 t1[1] = 40
      2 t1

TypeError: 'tuple' object does not support item assignment
```

In [ ]:
```
# tuple is immutable and list is mutable
```

In [83]:
```
sbi = (1234 , 'cizps')
```

```
sbi
```

Out[83]: (1234, 'cizps')

In [84]:
```
sbi[0] = 456
sbi
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[84], line 1
----> 1 sbi[0] = 456
      2 sbi

TypeError: 'tuple' object does not support item assignment
```

In [85]:
```
t1
```

Out[85]: (10, 20, 30)

In [86]:
```
t1 * 3
```

Out[86]: (10, 20, 30, 10, 20, 30, 10, 20, 30)

In [ ]:
```
# list and tuple completed
```

In [87]:
```
l7 = [10,20,40]
l7 * 2
```

Out[87]: [10, 20, 40, 10, 20, 40]

In [88]:
```
t1
```

Out[88]: (10, 20, 30)

In [ ]:
```
# SET
```

In [89]:
```
s = {}
s
```

Out[89]: {}

In [90]:
```
type(s)
```

Out[90]: dict

In [91]:
```
s1 = set()
s1
```

Out[91]: set()

In [92]:
```
s1 = {20,30,90,10}
s1
```

Out[92]: {10, 20, 30, 90}

In [ ]:
```
# set is ordered (ascending order)
```

In [93]:
```
type(s1)
```

Out[93]: set

In [ ]:
```
# {} -- by default it will considered as DICT
# {10,20} -- {} with value is take care as set
```

In [94]:
```
s1
```

Out[94]: {10, 20, 30, 90}

In [95]:
```
s1.add(100)
s1
```

Out[95]: {10, 20, 30, 90, 100}

In [98]:
```
s1.add(5)
s1
```

Out[98]: {5, 10, 15, 20, 30, 90, 100}

In [100…
```
s2 = {5,5.5,'nit',1+2j,True}
s2
```

Out[100… {(1+2j), 5, 5.5, True, 'nit'}

In [101…
```
print(s1)
print(s2)
```

```
{100, 5, 10, 15, 20, 90, 30}
{True, 5.5, 5, (1+2j), 'nit'}
```

In [102…
```
s3 = s2.copy()
s3
```

Out[102… {(1+2j), 5, 5.5, True, 'nit'}

In [ ]:
```
# indexing not allowed in set it will allow only list
```

In [111…
```
s3.pop(4)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[111], line 1
----> 1 s3.pop(4)

TypeError: set.pop() takes no arguments (1 given)
```

In [104…
```
s3.pop()
```

Out[104… True

In [105…
```
s3
```

Out[105… {(1+2j), 5, 5.5, 'nit'}

In [106…
```
s4 = {100, (3+4j), 'nit',5.7}
s4
```

Out[106… {(3+4j), 100, 5.7, 'nit'}

In [107…
```
s4.discard(3+4j)
```

In [108…
```
s4
```

Out[108… {100, 5.7, 'nit'}

In [109…
```
s4.discard(1000)
s4
```

Out[109… {100, 5.7, 'nit'}

In [110…
```
s4.remove(1000)
s4
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[110], line 1
----> 1 s4.remove(1000)
      2 s4

KeyError: 1000
```

In [ ]:
```
# remove gives an error , but discard will not give error
```

In [ ]:
```
# in set duplicates are not allowed
```

In [112…
```
s4
```

Out[112… {100, 5.7, 'nit'}

In [116…
```
s4.add(100)
s4
```

Out[116… {100, 5.7, 'nit'}

In [ ]:
```
# set operations  --->
# union
# intersection
```

In [117…
```
s7 = {1,2,3,4,5}
s8 = {4,5,6,7}
```

```
s9 = {7,8,9,10}
s7.union(s8)
```

Out[117…
```
{1, 2, 3, 4, 5, 6, 7}
```

In [118…
```
s7.union(s8,s9)
```

Out[118…
```
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
```

In [119…
```
s8 | s9
```

Out[119…
```
{4, 5, 6, 7, 8, 9, 10}
```

In [120…
```
s8 | s9 | s7
```

Out[120…
```
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
```

In [121…
```
print(s7)
print(s8)
print(s9)
```

```
{1, 2, 3, 4, 5}
{4, 5, 6, 7}
{8, 9, 10, 7}
```

In [ ]:

In [ ]: